APTECH COMPUTER EDUCATION

Techwiz 7 | Multi-Platform App Computing

PennyPal

A Smart Budgeting Companion for Students

Theme: NextGen BudgetBee

PROJECT DOCUMENTATION

Problem Definition - Solution - Site Map - System Design

Team Name	Team Outis
Mentor Name	Alee
Batch / Semester	5th semester
Aptech Center	Aptech Larkana

TEAM MEMBERS

Sr.	Student ID	Student Name
1	Student1614250	Zoya Falak
2	Student161424	Mohsin Ali Shaikh
3	Student1614243	Hasnain Shaikh

Table of Contents

1. Introduction 3
1.1 Project Overview 3
1.2 Purpose of the Document 3
1.3 Scope 3
1.4 Objectives 3
1.5 Target Users 3
2. Problem Definition 4
2.1 Problem Statement 4
2.2 Problems Identified 4
2.3 Existing Solutions and Their Limitations 4
3. Proposed Solution 5
3.1 Solution Overview 5
3.2 Problem-to-Solution Mapping 5
3.3 Key Features 5
3.4 What Makes PennyPal Different 5
4. Requirement Analysis 7
4.1 Functional Requirements 7
4.2 Non-Functional Requirements 7
4.3 Hardware and Software Requirements 8
5. Feasibility Study 9
6. System Design 10
6.1 System Architecture 10
6.2 Site Map 11
6.3 Use Case Diagram 11
6.4 Data Flow Diagrams 12
6.5 Entity Relationship Diagram 14
6.6 Data Dictionary 14
7. User Interface Design 17
7.1 Design Principles 17
7.2 Colour Palette 17
7.3 Typography 17
7.4 Key Screens 17
7.5 Navigation Flow 18
8. Technology Stack 19
9. Testing 20
9.1 Testing Approach 20

9.2 Sample Test Cases 20
10. Project Plan 21
11. Future Enhancements 22
12. Conclusion 23
Bibliography 24

1. Introduction

1.1 Project Overview

PennyPal is a multi-platform budgeting application designed specifically for students. It helps a student record income and expenses, set spending limits, save towards goals and understand where their money goes each month, all from a clean, friendly interface that runs on Android, iOS, web and desktop from a single Flutter codebase.

The project is developed for Aptech Techwiz 7 under the theme NextGen BudgetBee, which asks for a modern, engaging solution that builds healthy money habits in young users.

1.2 Purpose of the Document

This document describes the problem PennyPal addresses, the proposed solution, the requirements, the site map and the technical design of the system. It serves as the reference for developers, evaluators and stakeholders throughout the project.

1.3 Scope

In scope: user registration and login, income and expense tracking, category management, budgets with progress alerts, savings goals, bill reminders, reports and charts, financial tips, profile, currency and theme settings, offline use with cloud sync.

Out of scope (this release): real bank account linking, payment processing, investment tools and multi-user shared wallets.

1.4 Objectives

- Make daily expense tracking possible in under 10 seconds.
- Help students stay within a monthly budget through timely alerts.
- Encourage saving by turning goals into visible, trackable progress.
- Deliver one consistent experience across mobile, web and desktop.
- Keep the app usable offline and sync automatically when online.
- Teach basic financial literacy through short in-app tips.

1.5 Target Users

User Group	Description	Key Need
School / college students	Age 15-22, limited allowance or pocket money	Simple tracking and saving habit
University students	Age 18-26, part-time income, rent and tuition	Monthly budgeting and bill reminders
Young first-time earners	Interns and freelancers starting to manage income	Clear income vs expense overview

2. Problem Definition

2.1 Problem Statement

Most students receive money in irregular amounts and spend it without a plan. By the end of the month they cannot explain where their money went, miss bill due dates, and rarely save anything. They have no habit of budgeting because existing tools feel complicated, are built for adults with salaries, or show too much financial jargon.

2.2 Problems Identified

#	Problem	Effect on the Student
P1	No record of daily spending	Money disappears without awareness
P2	No spending limits or early warnings	Overspending before month end
P3	No clear savings target	Students never build a savings habit
P4	Forgotten bills and payments	Late fees and stress
P5	Complex, adult-oriented finance apps	Students abandon them quickly
P6	Poor financial literacy	Bad money decisions carried into adulthood
P7	Apps limited to one platform or need constant internet	Not usable on every device or in low connectivity

2.3 Existing Solutions and Their Limitations

Approach	Limitation
Notebook / paper record	Easy to forget, no totals, no charts, easily lost
Spreadsheets	Slow to enter on a phone, no reminders, needs skill to set up
General finance apps	Too complex, ads or paid features, not designed around student life
Bank apps	Only show bank transactions, not cash spending, no goal or budget guidance

3. Proposed Solution

3.1 Solution Overview

PennyPal replaces guesswork with a simple loop: record, review, adjust. A student adds a transaction in a few taps, sees instantly how it affects the budget, and gets gentle nudges before problems happen. Savings goals and short tips keep the student motivated.

3.2 Problem-to-Solution Mapping

Problem	PennyPal Solution	Module
P1 No record of spending	Quick-add income and expense with category, note and date	Transactions
P2 No limits or warnings	Monthly and weekly budgets with progress bars and 80% / 100% alerts	Budgets
P3 No savings target	Savings goals with deadline, contributions and progress ring	Savings Goals
P4 Forgotten bills	Bill reminders with due date, repeat option and notifications	Bill Reminders
P5 Complex apps	Minimal screens, bottom navigation, plain language, light and dark themes	UI / UX
P6 Low financial literacy	Daily tips and short money lessons	Financial Tips
P7 Platform and internet limits	Single Flutter codebase for all platforms, offline storage with cloud sync	Architecture

3.3 Key Features

- **Dashboard:** balance, income vs expense summary, spending chart and recent activity in one view.
- **Transactions:** add, edit, delete, search and filter by date, category or type.
- **Budgets:** set limits per category and track them live.
- **Savings Goals:** create goals, add contributions and watch progress.
- **Reports:** weekly and monthly charts, category breakdown, income vs expense trend.
- **Bill Reminders:** never miss a due date with repeat reminders.
- **Financial Tips:** bite-sized advice for students.
- **Personalisation:** currency selection and light / dark theme.
- **Offline First:** works without internet and syncs when connected.

3.4 What Makes PennyPal Different

- Built around student life, not adult salary budgeting.
- Friendly penny-and-bee identity that makes budgeting feel less like a chore.

- One codebase delivering a consistent experience on four platforms.
- Free of clutter: every screen has one clear job.

4. Requirement Analysis

4.1 Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow a user to register, log in, log out and reset a password.	High
FR-02	The system shall allow the user to add income and expense transactions with amount, category, date and note.	High
FR-03	The system shall allow the user to edit and delete transactions.	High
FR-04	The system shall provide default categories and allow custom categories.	Medium
FR-05	The system shall allow budgets to be set per category for a week or month.	High
FR-06	The system shall alert the user when spending reaches 80% and 100% of a budget.	High
FR-07	The system shall allow savings goals with target amount and deadline.	Medium
FR-08	The system shall allow contributions to be added to a goal and show progress.	Medium
FR-09	The system shall display charts and reports for selected periods.	High
FR-10	The system shall send reminders for upcoming bills.	Medium
FR-11	The system shall search and filter transactions.	Medium
FR-12	The system shall allow currency and theme (light / dark) selection.	Low
FR-13	The system shall store data in Cloud Firestore, cache it on the device for offline use and sync automatically when online.	High
FR-14	The system shall show financial tips to the user.	Low

4.2 Non-Functional Requirements

Category	Requirement
Performance	App launches in under 3 seconds; adding a transaction completes in under 1 second.
Usability	A new user can add their first expense within one minute without instructions.
Security	Credentials are managed by Firebase Authentication; data is transmitted over HTTPS; Firestore Security Rules let each user access only their own data.
Reliability	No data loss on crash; offline entries are queued and synced later.
Compatibility	Runs on Android, iOS, modern web browsers and desktop (Windows).
Scalability	Cloud backend supports growth in users without redesign.
Maintainability	Layered architecture with reusable widgets and clear folder structure.
Accessibility	Readable contrast, scalable text and touch targets of at least 48 px.

4.3 Hardware and Software Requirements

Item	Development	End User
Processor	Intel Core i5 or equivalent	Any modern smartphone / PC processor
RAM	8 GB minimum	2 GB minimum
Storage	10 GB free	100 MB free
Operating System	Windows 10/11	Android 8+, iOS 13+, Windows 10+, any modern browser
Software	Flutter SDK, Dart, VS Code / Android Studio, Git	None (install app or open in browser)

5. Feasibility Study

Type	Assessment
Technical	Feasible. Flutter and Dart are mature, well documented and support all target platforms from one codebase. Required packages for charts, local storage and notifications are available.
Economic	Feasible. All tools are free or open source and the cloud backend has a free tier suitable for the project scale.
Operational	Feasible. The app targets a simple daily task and needs no training. Its interface follows familiar mobile patterns.
Schedule	Feasible. The scope is divided into small modules that can be built and tested in parallel within the Techwiz timeline.

6. System Design

6.1 System Architecture

PennyPal follows a layered architecture. The presentation layer contains Flutter screens and widgets; state management keeps the UI in sync with data; services hold business rules such as budget calculations; and repositories abstract local and remote storage.

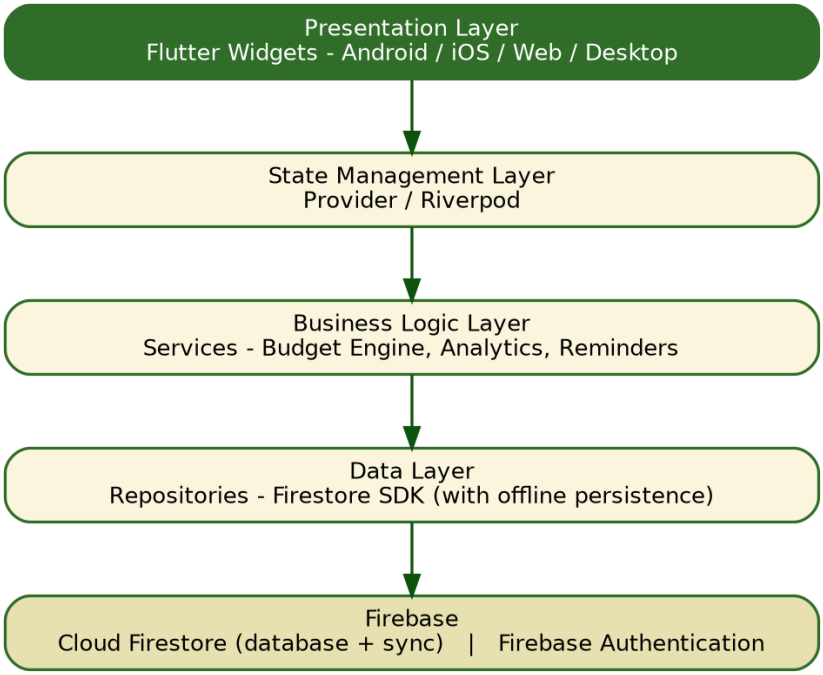


Figure 6.1: PennyPal layered architecture

6.2 Site Map

The site map shows every screen in the application and how the user reaches it. After the splash, onboarding and login flow, the user lands on a home shell with five bottom-navigation tabs: Dashboard, Transactions, Budgets, Savings Goals and More.

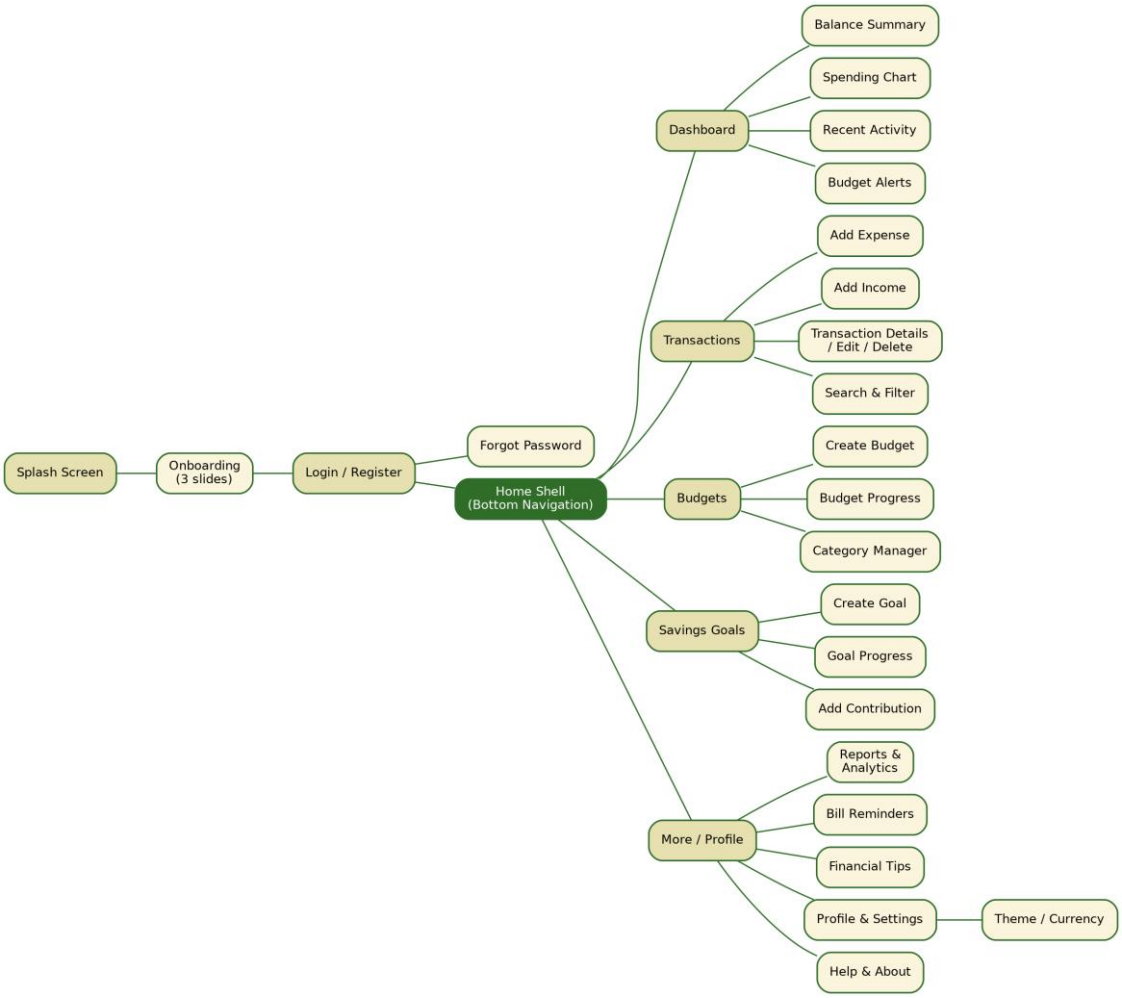


Figure 6.2: PennyPal site map

6.3 Use Case Diagram

The student is the primary actor. Data synchronisation with the cloud backend is included whenever a transaction or goal contribution is saved.

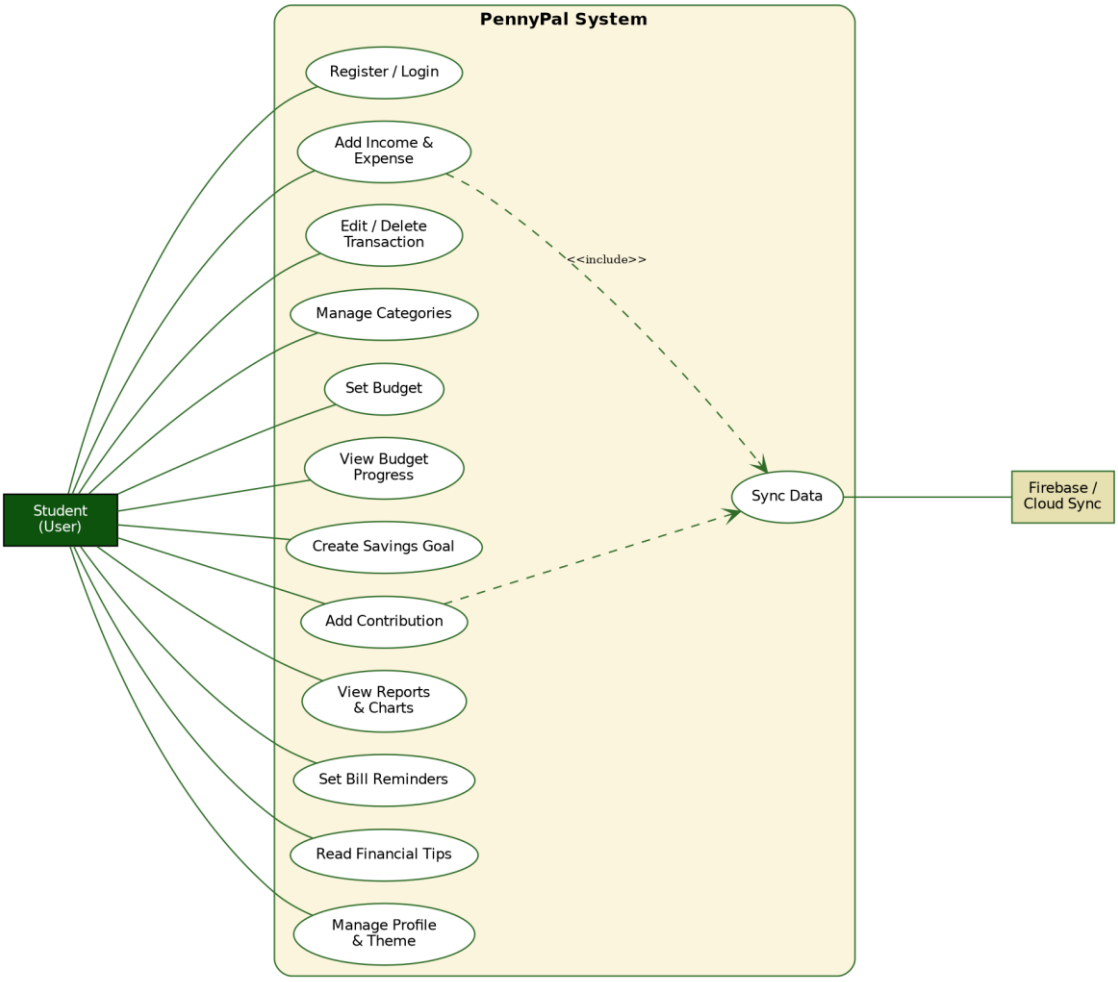


Figure 6.3: Use case diagram

6.4 Data Flow Diagrams

Level 0 (Context Diagram)

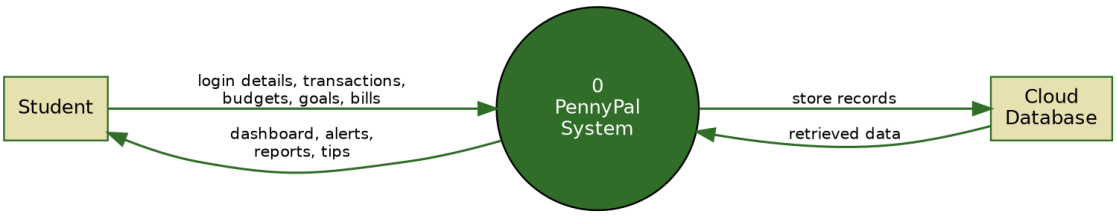


Figure 6.4: DFD Level 0

Level 1

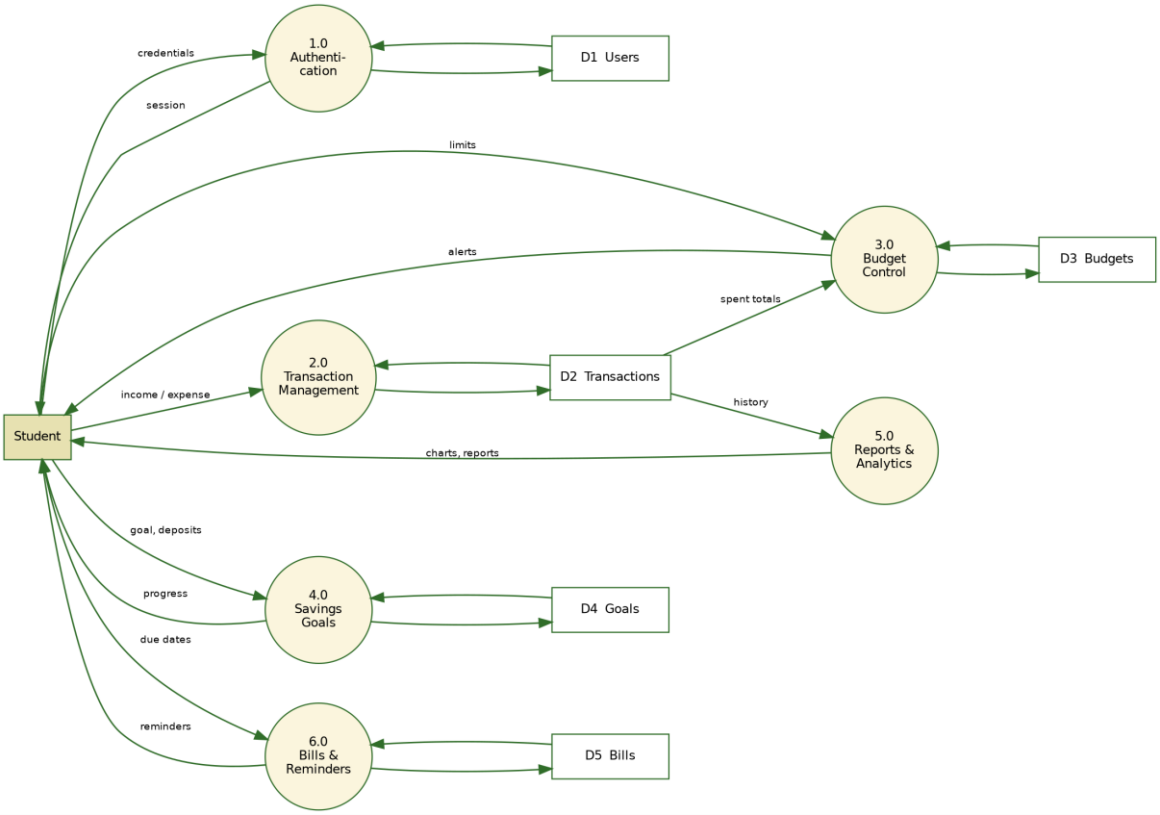


Figure 6.5: DFD Level 1

6.5 Entity Relationship Diagram

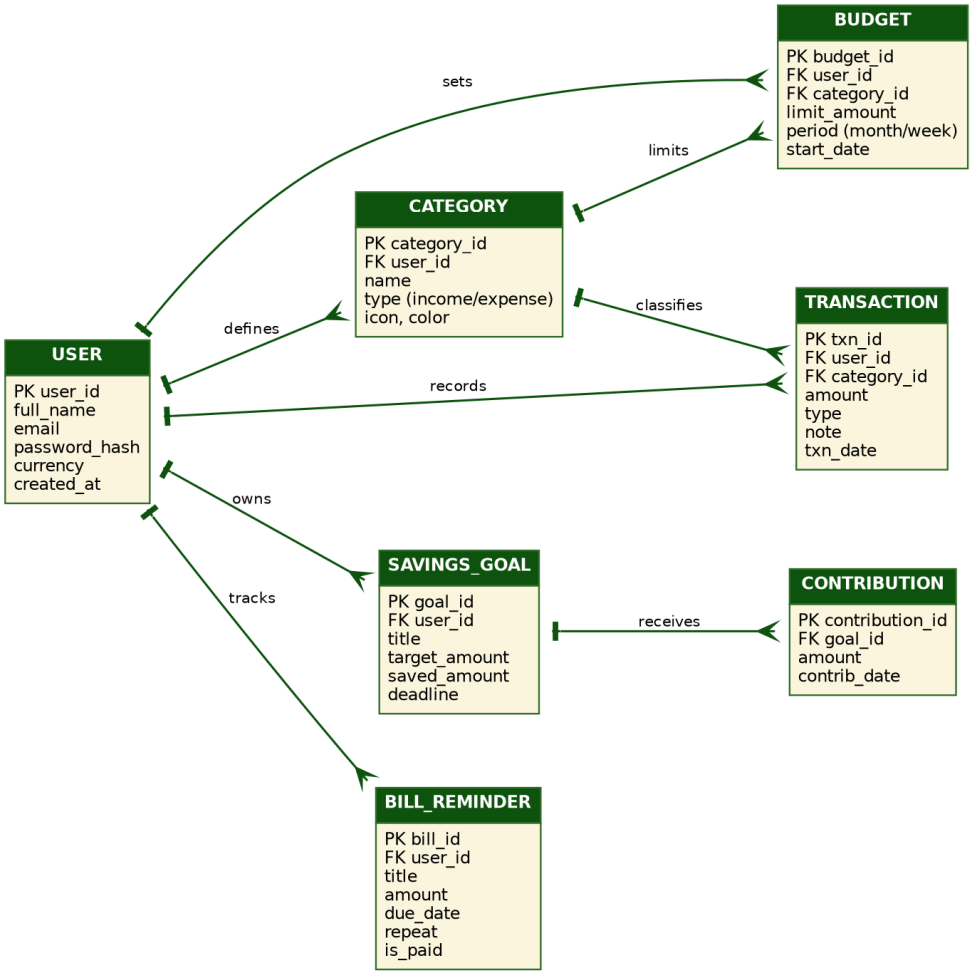


Figure 6.6: Entity relationship diagram

6.6 Data Dictionary

PennyPal stores data in Cloud Firestore. Each entity below is a Firestore collection (transactions, budgets, goals and bills are kept under the owning user), and each row is a field of a document. Passwords are not stored in the database; they are handled by Firebase Authentication.

USER

Field	Type	Constraint / Description
user_id	String	Primary key (Firebase Auth UID)
full_name	String(80)	Not null
email	String(120)	Unique, not null
currency	String(5)	Default PKR
created_at	DateTime	Auto generated

CATEGORY

Field	Type	Constraint / Description
category_id	String / UUID	Primary key
user_id	String	Foreign key to USER (null for defaults)
name	String(40)	Not null
type	Enum	income / expense
icon, color	String	UI display values

TRANSACTION

Field	Type	Constraint / Description
txn_id	String / UUID	Primary key
user_id	String	Foreign key to USER
category_id	String	Foreign key to CATEGORY
amount	Decimal(12,2)	Greater than 0
type	Enum	income / expense
note	String(200)	Optional
txn_date	DateTime	Not null

BUDGET

Field	Type	Constraint / Description
budget_id	String / UUID	Primary key
user_id	String	Foreign key to USER
category_id	String	Foreign key to CATEGORY
limit_amount	Decimal(12,2)	Greater than 0
period	Enum	week / month
start_date	Date	Not null

SAVINGS_GOAL

Field	Type	Constraint / Description
goal_id	String / UUID	Primary key
user_id	String	Foreign key to USER
title	String(60)	Not null

Field	Type	Constraint / Description
target_amount	Decimal(12,2)	Greater than 0
saved_amount	Decimal(12,2)	Default 0
deadline	Date	Optional

CONTRIBUTION

Field	Type	Constraint / Description
contribution_id	String / UUID	Primary key
goal_id	String	Foreign key to SAVINGS_GOAL
amount	Decimal(12,2)	Greater than 0
contrib_date	DateTime	Not null

BILL_REMINDER

Field	Type	Constraint / Description
bill_id	String / UUID	Primary key
user_id	String	Foreign key to USER
title	String(60)	Not null
amount	Decimal(12,2)	Optional
due_date	Date	Not null
repeat	Enum	none / weekly / monthly
is_paid	Boolean	Default false

7. User Interface Design

7.1 Design Principles

- Clarity: one main action per screen.
- Speed: the add-transaction button is always reachable.
- Consistency: the same components and spacing on every platform.
- Warmth: a friendly penny-and-bee personality without clutter.
- Comfort: a well-balanced dark mode without glow effects.

7.2 Colour Palette

The light theme uses a natural green and cream palette. The dark theme uses a midnight navy base with penny gold accents and flat, glow-free surfaces.

Role	Light Theme	Hex
Background	Soft cream	#FBF5DD
Surface / Card	Sand	#E7E1B1
Primary	Leaf green	#306D29
Primary Dark / Text accent	Forest green	#0D530E

Role	Dark Theme	Note
Background	Midnight navy	Deep, low-brightness base
Surface / Card	Lighter navy	Flat surface, subtle border, no glow
Accent	Penny gold	Buttons, highlights, active tab
Text	Warm off-white	High contrast on navy

7.3 Typography

A clean sans-serif typeface (Poppins for headings, Inter for body text) with a clear size scale: 28 / 20 / 16 / 14 / 12 px. Amounts are shown in a bold weight for quick scanning.

7.4 Key Screens

Screen	Purpose	Main Elements
Splash / Onboarding	Introduce the app	Logo, three intro slides, Get Started button
Login / Register	Authenticate the user	Email, password, validation messages, forgot password link
Dashboard	Financial snapshot	Balance card, income / expense summary, chart, recent list

Screen	Purpose	Main Elements
Transactions	List and manage records	Search bar, filters, grouped list, add button
Add Transaction	Fast entry	Amount keypad, category chips, date, note, Save
Budgets	Set and monitor limits	Category budgets, progress bars, alert colours
Savings Goals	Track goals	Goal cards, progress ring, add contribution
Reports	Analyse spending	Pie chart, monthly bar chart, period selector
Bill Reminders	Avoid missed bills	Upcoming list, mark as paid, repeat option
Profile & Settings	Personalise	Currency, theme toggle, notifications, logout

7.5 Navigation Flow

Splash, Onboarding and Login lead to the Home shell. From there the user switches between the five tabs using bottom navigation. A floating add button on the Dashboard and Transactions screens opens the Add Transaction screen in one tap. Back navigation always returns to the previous tab state.

8. Technology Stack

Layer	Technology	Reason
Framework	Flutter (Dart)	One codebase for Android, iOS, web and desktop
State Management	Provider / Riverpod	Simple, testable, reactive UI
Database	Cloud Firestore	NoSQL cloud database with real-time sync
Offline Support	Firestore offline persistence	Built-in local cache, syncs automatically when online
Authentication	Firebase Authentication	Secure email login and password reset
Charts	fl_chart	Attractive, customisable charts
Notifications	flutter_local_notifications	Budget alerts and bill reminders
Design Prototype	HTML prototype and Figma-style mockups	Visual reference for UI
Version Control	Git and GitHub	Collaboration and history
Tools	VS Code, Android Studio, Antigravity	Development and AI-assisted implementation

9. Testing

9.1 Testing Approach

Testing covers unit tests for business logic (budget and total calculations), widget tests for key screens, and manual functional testing on Android, web and desktop.

9.2 Sample Test Cases

ID	Test Case	Expected Result	Status
TC-01	Register with valid details	Account created and user logged in	Pass
TC-02	Login with wrong password	Error message shown, access denied	Pass
TC-03	Add expense of 500 in Food	Transaction listed, balance reduced by 500	Pass
TC-04	Add expense with empty amount	Validation error, not saved	Pass
TC-05	Spend 80% of a budget	Warning alert shown	Pass
TC-06	Exceed a budget	Over-budget alert, progress bar turns red	Pass
TC-07	Add contribution to a goal	Saved amount and progress update	Pass
TC-08	Delete a transaction	Removed and totals recalculated	Pass
TC-09	Use app offline, then reconnect	Entries synced to cloud	Pass
TC-10	Switch to dark theme	All screens change theme without restart	Pass

Status values are to be confirmed after final test execution.

10. Project Plan

Phase	Activities	Output
1. Analysis	Study SRS, identify problems and requirements	Requirement list
2. Design	Site map, database design, UI mockups, colour palette	Design documents and prototype
3. Development	Build authentication, transactions, budgets, goals, reports, reminders	Working app modules
4. Integration	Connect local storage and cloud sync, apply themes	Integrated application
5. Testing	Unit, widget and manual testing, bug fixing	Test report
6. Documentation & Submission	Prepare documentation and demo	Final submission

11. Future Enhancements

- AI-based spending insights and saving suggestions.
- Receipt scanning with OCR.
- Shared budgets for roommates and family.
- Bank and wallet integration (Easypaisa, JazzCash, bank APIs).
- Multi-language support including Urdu.
- Gamified challenges and streak rewards.
- Export reports as PDF and CSV.

12. Conclusion

PennyPal addresses a real everyday problem: students do not know where their money goes. By combining fast transaction entry, budgets with alerts, savings goals, bill reminders and clear reports in a simple multi-platform app, it builds healthy financial habits early. The layered Flutter architecture, offline-first storage and clean design make the system reliable, maintainable and ready for future growth.

Bibliography

- Flutter Documentation - https://docs.flutter.dev
- Dart Language Guide - https://dart.dev/guides
- Firebase Documentation - https://firebase.google.com/docs
- Material Design Guidelines - https://m3.material.io
- fl_chart package - https://pub.dev/packages/fl_chart
- Aptech Techwiz 7 Software Requirement Specification (SRS) for PennyPal